



Chapter S1C: Sorting Algorithms

CONTENT

No.	Topic	Syllabus Guide
1	Sorting Fundamentals	1.2.1 – 1.2.2
2	Insertion Sort	1.2.1 – 1.2.2
3	Bubble Sort	1.2.1 – 1.2.2
4	Quicksort	1.2.1 – 1.2.2
5	Merge Sort	1.2.1 – 1.2.2
6	Time Complexity	1.2.5
7	Video References	
8	Appendix	
	8.1 Analysis of Quicksort Efficiency	
	8.2 Analysis of Merge Sort Efficiency	

1. SORTING FUNDAMENTALS

Sorting is the process of arranging a collection of data items—such as numbers, strings, or records—in a specific order, based on a defined ordering criterion. The most common types of orderings are:

- Numerical order (e.g., from smallest to largest)
- Lexicographical (alphabetical) order

Sorting can be performed on an entire item or based on a specific part (or key) of each item. For example, a record containing a student's name, class, and exam score can be sorted based on the exam score.

Why is sorting important?

Sorting data makes it easier to:

- Search for items efficiently (e.g., binary search)
- Present information in a readable form
- Process data in a meaningful sequence

Real-world examples of sorting

- **File Explorer:** Files and folders can be sorted by name, date, size, or file type.
- **Email applications:** Emails can be sorted by date received, sender, or subject.
- **Telephone directory:** Entries are sorted by name to make it easier to search for a person.

1.1 Operations Involved In Sorting

Sorting algorithms usually involve two core operations: **comparison** and **swapping**.

Comparison

This determines the order of elements based on the sorting criterion. For instance,

- Sorting numbers in ascending order: compare which is smaller.
- Sorting strings alphabetically: compare character codes (e.g., ASCII).

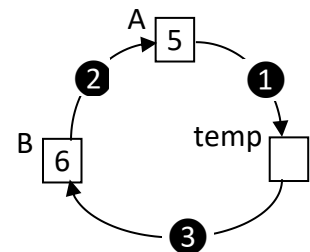
Sorting algorithms often perform **many comparisons** to determine the correct order of all elements.

Swapping

This moves elements into their correct positions by exchanging values.

For example, `SWAP (A, B)` requires copy operations:

- `temp ← A` (assign value of A to temp)
- `A ← B` (assign value of B to A)
- `B ← temp` (Assign value of temp to B)



1.2 In-Place and Not In-Place Sorting

In-Place Sorting

An **in-place algorithm** sorts the data within the original data structure using only a **constant amount of extra memory** (i.e. $O(1)$ space). The sorting is done by rearranging the elements directly. Sorting in-place is common when the **data fits into main memory (internal sorting)**. Examples include **insertion sort**, **bubble sort**, and **quicksort**.

Example – Reversing an Array In-Place

```
//To reverse the items of an array and
//store the reversed results back into itself
PROCEDURE ReverseArray(arr: ARRAY of INTEGER, N: INTEGER):
    //N is the size of array
    DECLARE temp: INTEGER
    FOR i = 1 TO INT(N / 2)
        temp ← arr[i]
        arr[i] ← arr[N - i + 1]
        arr[N - i + 1] ← temp
    ENDFOR
ENDPROCEDURE

//main program
arr ← [1, 2, 3, 7, 8, 9]
N ← LENGTH(arr)
OUTPUT("Original array is", arr)
ReverseArray(arr, N);
OUTPUT("Reversed array is", arr)
```

Not In-Place Sorting

A **not in-place algorithm** requires **additional memory** to perform the sorting – usually by copying the data into another structure. This is often necessary when dealing with **large datasets** that do not fit into memory all at once.

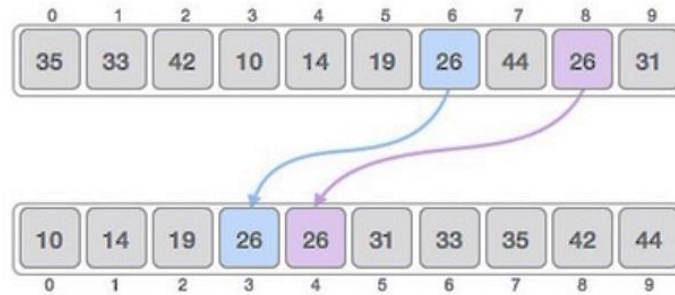
Sorting with additional memory is called **external sorting**. It is used by algorithms like **merge sort**, especially for large datasets stored on disk.

Example – Reversing an Array using extra space

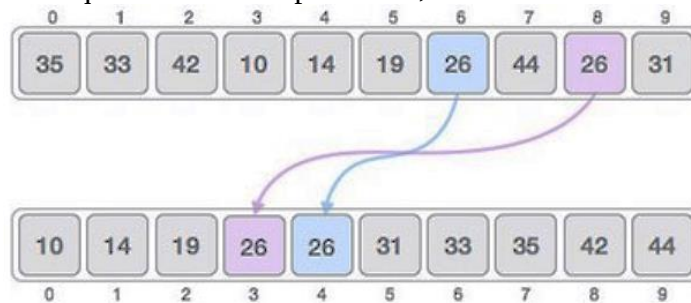
```
//To reverse the items of an array to a new array in function
//and store the reversed results back into itself
PROCEDURE ReverseArray(arr: ARRAY of INTEGER, N: INTEGER):
    //N is the size of array
    //Creates a copy of the array to store its reversed elements
    DECLARE rev: ARRAY of INTEGER[1..N]
    FOR i ← 1 TO N
        rev[N - i + 1] ← arr[i]
    ENDFOR
    //Copies reversed elements back to arr as results
    FOR i ← 1 TO N
        arr[i] ← rev[i]
    ENDFOR
ENDPROCEDURE
```

1.3 Stable and Unstable Sorting

A sorting algorithm is stable if it preserves the relative order of items with equal keys.



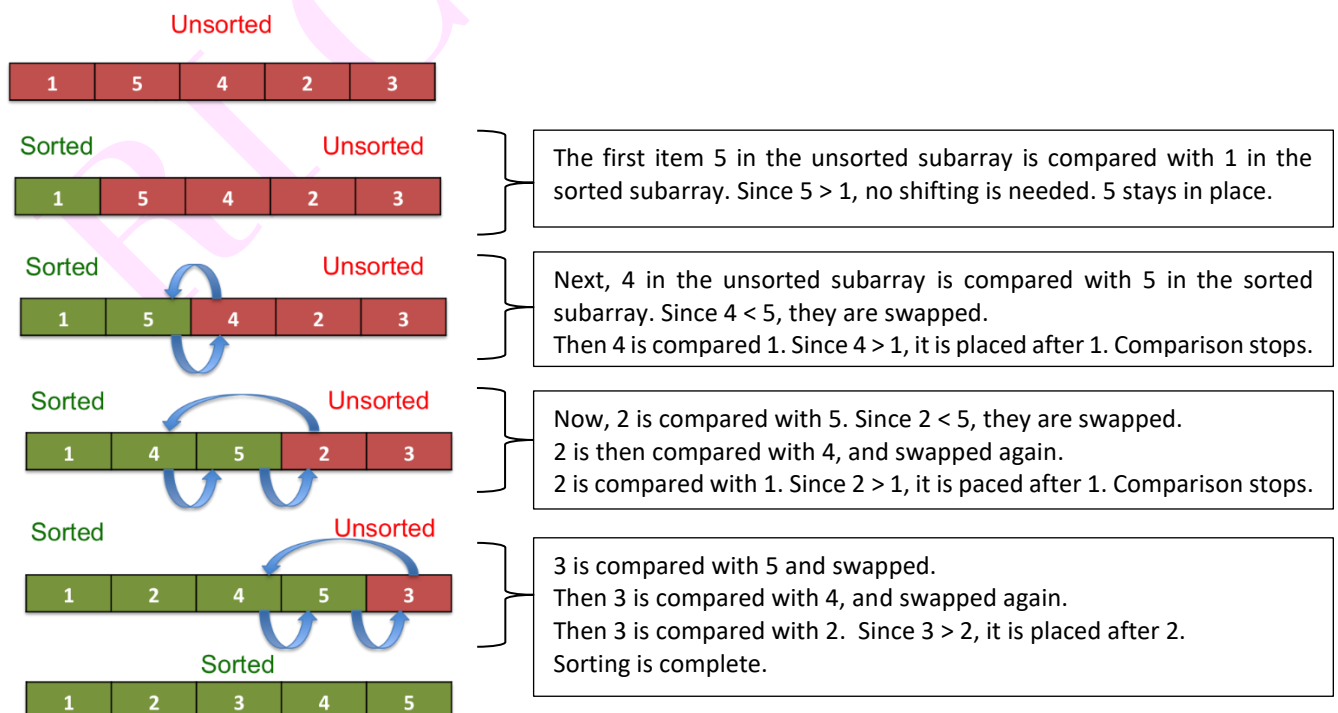
If the relative order of equal items is not preserved, the sort is unstable.



Insertion sort, bubble sort and merge sort are stable sorting algorithms by nature, while quicksort is unstable.

2. INSERTION SORT

Insertion sort algorithm sorts *one data item at a time*. The array is “split” into a **sorted subarray** and an **unsorted subarray**. The algorithm takes one item from the unsorted subarray and “**inserts**” it in the correct position in the sorted subarray.



2.1 Key Steps

1. Start by considering the first element as the **sorted subarray**, and the rest as **unsorted**.
2. For each iteration, take the first element of the unsorted subarray (starting from the second element of the list).
3. Move **backward** through the sorted subarray, comparing and shifting elements to the right until the correct position for the current element is found.
4. Insert the element into its correct position.
5. Repeat until all elements have been inserted into the sorted subarray – i.e. when the unsorted subarray becomes empty.

Each iteration increases the size of the sorted subarray by one element and decreases the size of the unsorted subarray by one.

2.2 Pseudocode for Insertion Sort (Ascending Order)

```

PROCEDURE INSERTIONSORT(arr: ARRAY)
  FOR index ← 1 TO LENGTH(arr)-1 //base-0 indexing
    unsortedValue ← arr[index]
    current ← index - 1
    WHILE current >= 0 AND unsortedValue < arr[current]
      arr[current + 1] ← arr[current]
      current ← current - 1
    ENDWHILE
    arr[current+1] ← unsortedValue
  ENDFOR
ENDPROCEDURE

```

2.3 Trace Table of Insertion Sort

Given arr = [9,5,8,12,3,7,11], sort the array in ascending order INSERTIONSORT(arr):

	index	unsorted Value	current	current >= 0 and unsortedValue < arr[current]	[0]	[1]	[2]	[3]	[4]	[5]	[6]
arr					9	5	8	12	3	7	11
	1	5	0	True		9					
			-1	False	5						
	2	8	1	True			9				
			0	False		8					
	3	12	2	False				12			
	4	3	3	True					12		
			2	True				9			
			1	True			8				
			0	True		5					
			-1	False	3						
	5	7	4	True						12	
			3	True					9		
			2	True				8			
			1	False			7				
	6	11	5	True							12
			4	False						11	
Sorted					3	5	7	8	9	11	12

3. BUBBLE SORT

Bubble sort repeatedly steps through the list, compares adjacent items, and swaps them if they are in the wrong order. With each pass, the largest (or smallest) value “bubbles” to its correct position.

3.1 Key Steps

1. Traverse the list and compare each pair of adjacent items.
2. If a pair is in the wrong order, swap them.
3. After one complete pass, the largest item is at the end.
4. Repeat the process for the remaining items, reducing the number of comparisons after each pass.

Example of a bubble sort that stops when no swaps are needed during a pass.

Bubble sort on [4, 7, 3, 1, 6] (ascending order):			
<u>1st pass: 4 7 3 1 6</u>		<u>3rd pass: 3 1 4 6 7</u>	
• 4 7 3 1 6	→ No Swap	• 3 1 4 6 7	→ Swap → 1 3 4 6 7
• 4 7 3 1 6	→ Swap → 4 3 7 1 6	• 1 3 4 6 7	→ No Swap
• 4 3 7 1 6	→ Swap → 4 3 1 7 6	• 1 3 4 6 7	→ No Swap
• 4 3 1 7 6	→ Swap → 4 3 1 6 7	• 1 3 4 6 7	→ No Swap
<u>2nd pass: 4 3 1 6 7:</u>		<u>4th pass: 1 3 4 6 7</u>	
• 4 3 1 6 7	→ Swap → 3 4 1 6 7	• 1 3 4 6 7	→ No Swap
• 3 4 1 6 7	→ Swap → 3 1 4 6 7	• 1 3 4 6 7	→ No Swap
• 3 1 4 6 7	→ No Swap	• 1 3 4 6 7	→ No Swap
• 3 1 4 6 7	→ No Swap	• 1 3 4 6 7	→ No Swap (Stop)

3.2 Pseudocode for Basic Bubble Sort (Ascending Order)

```

PROCEDURE BUBBLESORT(arr: ARRAY OF INTEGER)
  DECLARE temp: INTEGER
  FOR i = 0 TO LENGTH(arr) - 2    //base-0 indexing
    FOR j = 0 TO (LENGTH(arr) - i - 2)
      IF arr[j] > arr[j + 1] THEN
        //Swap the items in array
        temp ← arr[j]
        arr[j] ← arr[j + 1]
        arr[j + 1] ← temp
      ENDIF
    ENDFOR
  ENDFOR
ENDPROCEDURE

```

3.3 Pseudocode for an Optimised Bubble Sort Using a Flag

This version stops early if no swaps are made in a pass, improving efficiency.

```

PROCEDURE BUBBLESORT(arr: ARRAY OF INTEGER)
  //FLAG keeps track if a swap is performed in a pass
  DECLARE temp: INTEGER
  DECLARE flag: BOOLEAN
  flag ← TRUE

  WHILE flag = TRUE
    flag ← FALSE
    FOR i = 0 TO LENGTH(arr)-2 //base-0 indexing
      IF arr[i] > arr[i+1] THEN
        temp ← arr[i]
        arr[i] ← arr[i+1]
        arr[i+1] ← temp
        flag ← TRUE //Indicates a swap performed
      ENDIF
    ENDFOR
  ENDWHILE
ENDPROCEDURE

```

3.4 Trace Table of Bubble Sort

Given this arr storing characters, sort in descending order:

[0]	[1]	[2]	[3]	[4]	[5]
K	C	L	B	M	D

Trace table:

Pass	arr[0]	arr[1]	arr[2]	arr[3]	arr[4]	arr[5]
	K	C	L	B	M	D
	K	L	C	B	M	D
	K	L	C	B	M	D
	K	L	C	M	B	D
1	K	L	C	M	D	B
	L	K	C	M	D	B
	L	K	C	M	D	B
	L	K	M	C	D	B
2	L	K	M	D	C	B
	L	K	M	D	C	B
	L	M	K	D	C	B
3	L	M	K	D	C	B
	M	L	K	D	C	B
4	M	L	K	D	C	B
5	M	L	K	D	C	B

Note:

Bubble sort for descending order changes the comparison to $\text{arr}[i] < \text{arr}[i + 1]$.

4. QUICKSORT

Quicksort is a sorting algorithm based on the **divide and conquer** paradigm, similar to merge sort. It works by breaking down a problem into smaller but similar sub-problems and then combining their solutions to solve the original problem.

4.1 How Quicksort Works

1. Choose an element from the array as the **pivot**.
2. Rearrange (partition) the elements of the array so that:
 - All elements less than the pivot appear to its left.
 - All elements greater than the pivot appear to its right.
3. Recursively apply the same process to the left and right sub-arrays.

4.2 Quicksort Pseudocode

```
PROCEDURE Quicksort (arr:ARRAY OF INTEGER, first:INTEGER, last:INTEGER)
  DECLARE pivotIndex: INTEGER
  IF first < last THEN
    pivotIndex = Partition(arr, first, last)
    Quicksort(arr, first, pivotIndex-1)
    Quicksort(arr, pivotIndex+1, last)
  ENDIF
ENDPROCEDURE
```

4.3 The Partition Algorithm

The purpose of the Partition procedure is to:

- Select a pivot (usually the first element).
- Reorganise the array so that all elements smaller than the pivot are on the left, and all elements larger are on the right.
- Return the final position of the pivot after the partition.

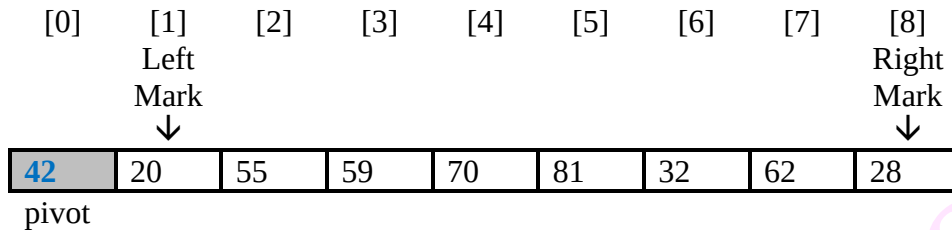
Steps:

1. Choose the first element as the pivot.
2. **Set two markers:**
 - LeftMark pointing to the element just after the pivot.
 - RightMark pointing to the last element of the array.
3. Repeat the following:
 - Move LeftMark right while the value is less than or equal to the pivot.
 - Move RightMark left while the value is greater than or equal to the pivot.
 - If LeftMark <= RightMark, swap the elements at these positions.
4. When LeftMark > RightMark, swap the pivot with the element at RightMark.
5. Return RightMark – the index where the pivot is now correctly placed.

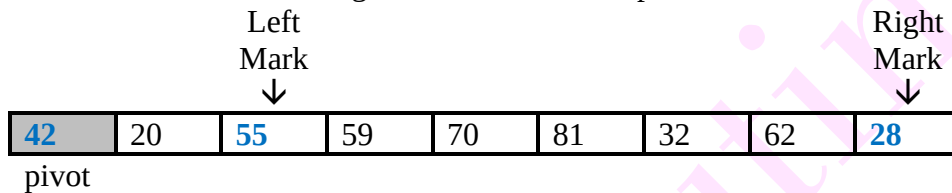
4.4 Worked Example of Partition Algorithm

Consider the array [42, 20, 55, 59, 70, 81, 32, 62, 28]

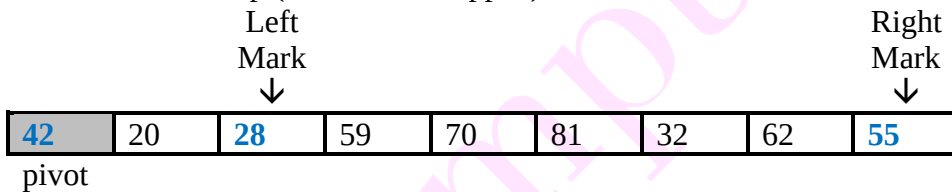
- Pivot: 42 (first element)
- Initial positions: LeftMark at index 1 (20), RightMark at index 8.



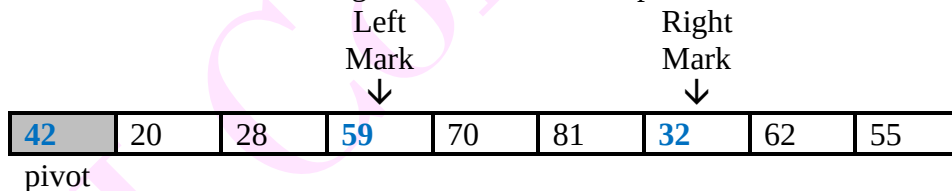
- LeftMark moves to 55, RightMark to 28 → Swap 55 and 28.



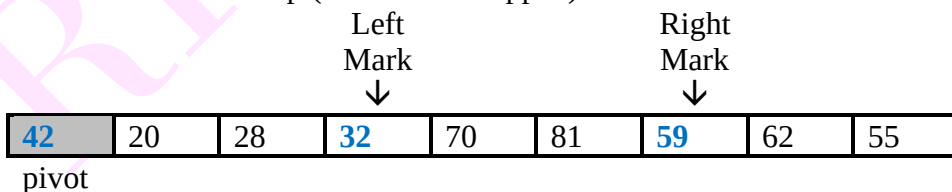
- After the first swap (55 and 28 swapped):



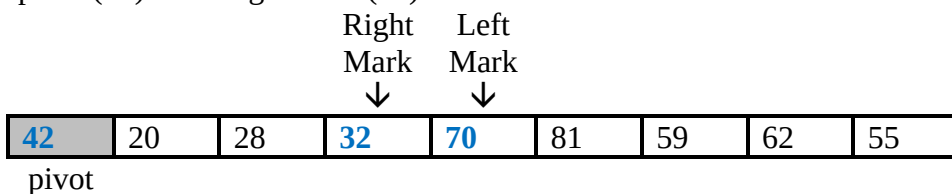
- LeftMark moves to 59, RightMark to 32 → Swap 59 and 32.



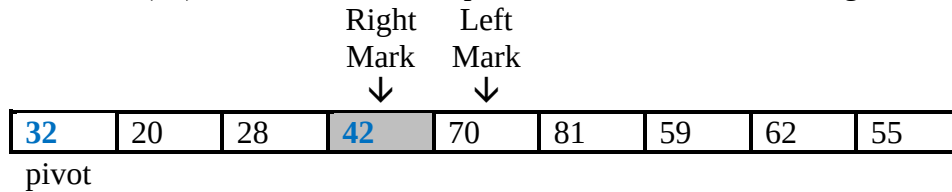
- After the second swap (59 and 32 swapped):



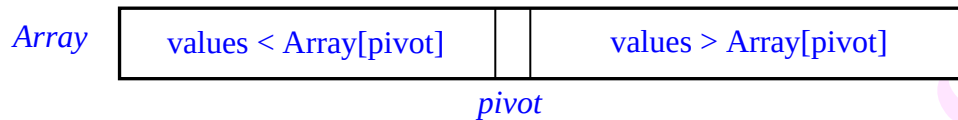
- LeftMark moves to 70, RightMark to 32 → Since LeftMark > RightMark, swap pivot (42) with RightMark (32).



- The pivot (42) is now in its correct position at index 3. Return RightMark.



After a call to the partition algorithm, pivot is selected and returned from the function. Array is divided into 2 sub-arrays, with values in the left sub-array smaller than the value at pivot, and values in the right sub-array larger than the value at pivot.



Quicksort algorithm is then applied to the left sub-array and right sub-array recursively.

4.5 Modular Quicksort and Partition (Pseudocode)

```

FUNCTION Partition(Arr, First, Last) RETURNS INTEGER
    PivotValue ← Arr[First]
    LeftMark ← First + 1
    RightMark ← Last
    Done ← FALSE

    WHILE Done = FALSE
        WHILE LeftMark ≤ RightMark AND Arr[LeftMark] ≤ PivotValue DO
            LeftMark ← LeftMark + 1
        ENDWHILE
        WHILE LeftMark ≤ RightMark AND Arr[RightMark] ≥ PivotValue DO
            RightMark ← RightMark - 1
        ENDWHILE
        IF LeftMark > RightMark THEN
            Done ← TRUE
        ELSE
            SWAP(Arr[LeftMark], Arr[RightMark])
        ENDIF
    ENDWHILE

    SWAP(Arr[First], Arr[RightMark])
    RETURN RightMark //Index of the pivot element that is in its correct position
ENDFUNCTION

FUNCTION Quicksort(Array, First, Last) RETURNS ARRAY
    IF First < Last THEN
        SplitPoint ← Partition(Array, First, Last)
        Quicksort(Array, First, SplitPoint - 1)
        Quicksort(Array, SplitPoint + 1, Last)
    ENDIF
    RETURN Array
ENDFUNCTION

```

```
// Main program
Scores ← [56, 87, 34, 63, 95, 23, 54, 23, 57, 65, 35, 86, 45, 92, 48, 63, 76]
SortedScores ← Quicksort(Scores, 1, LENGTH(Scores)) // 1-based indexing
```

Note: If implementing this in Python which uses 0-based indexing, the call should be:

```
SortedScores = Quicksort(Scores, 0, len(Scores)-1) // 0-based indexing
```

4.6 Choosing the Pivot

Different implementations of Quicksort may choose the pivot in different ways:

- First element (as above)
- Last element
- Middle element
- Random element

5. MERGE SORT

Merge Sort is a sorting method that uses a technique called **divide and conquer**. Instead of trying to sort everything at once, it breaks the problem into smaller parts, solves each part, and then puts everything back together in order.

Divide Phase:

- The list is split into two halves.
- Each half is split again and again, until you end up with many small lists that each have only one item.
- A list with one item is already sorted!

Conquer (Merge) Phase:

- Now, you start combining the small lists back together.
- As you merge, you compare the items from each list and place them in the right order.
- You keep merging until all the little lists become one big sorted list.

5.1 Merge Sort Algorithm

1. If the list contains only one element, it is already sorted – return it.
2. Otherwise:
 - Divide the list into two halves.
 - Recursively apply merge sort to each half.
 - Merge the two sorted halves into one sorted list.

5.2 Merge Sort Pseudocode

```
FUNCTION MergeSort(arr: ARRAY[1:N] of INTEGERS) RETURNS ARRAY
    IF N = 1 THEN
        RETURN arr
    ENDIF

    DECLARE list1: ARRAY[1: INT(N/2)] of INTEGER
    DECLARE list2: ARRAY[INT(N/2)+1:N] of INTEGER
    list1 ← MergeSort(list1)
    list2 ← MergeSort(list2)

    RETURN Merge(list1, list2)
ENDFUNCTION
```

5.3 Merge Pseudocode

```
FUNCTION Merge(left, right: ARRAY of INTEGERS) RETURNS ARRAY
    DECLARE i, j, k: INTEGER
    DECLARE combine: ARRAY[1:LENGTH(left)+LENGTH(right)] OF INTEGERS
    i ← 1
    j ← 1
    k ← 1

    WHILE i <= LENGTH(left) AND j <= LENGTH(right)
        IF left[i] > right[j] THEN
            combine[k] ← right[j]
            j ← j + 1
        ELSE
            combine[k] ← left[i]
            i ← i + 1
        ENDIF
        k ← k + 1
    ENDWHILE

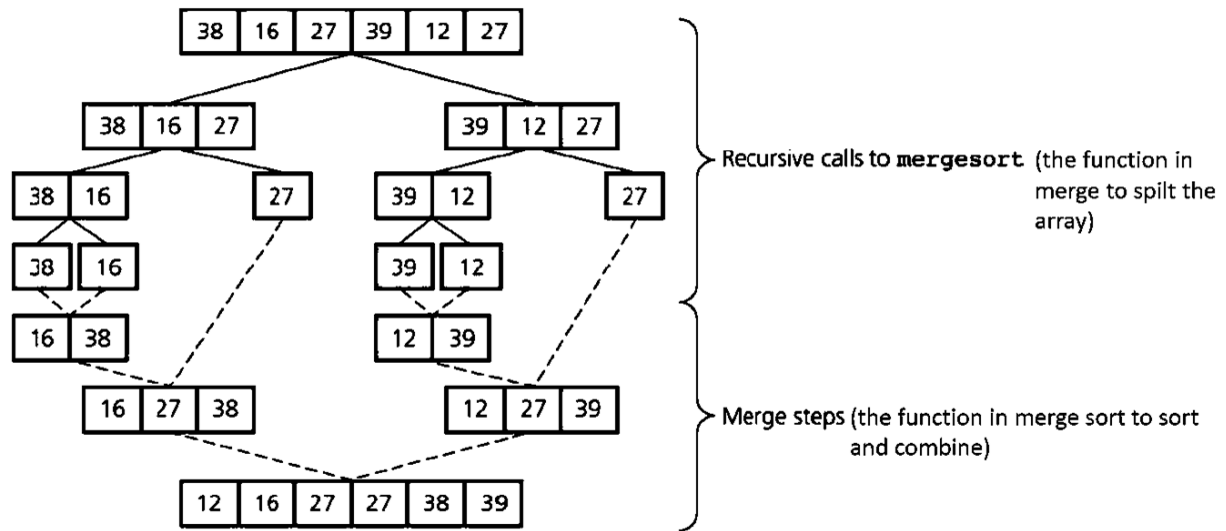
    WHILE i <= LENGTH(left) //add remaining elements from left to combine
        combine[k] ← left[i]
        k ← k + 1
        i ← i + 1
    ENDWHILE

    WHILE j <= LENGTH(right) //add remaining elements from right to combine
        combine[k] ← right[j]
        k ← k + 1
        j ← j + 1
    ENDWHILE

    RETURN combine
ENDFUNCTION
```

5.4 Example of Merge Sort

Apply on an array [38,16,27,39,12,27]:



6. TIME COMPLEXITY

Computer programs are often designed to solve complex problems or process large volumes of data. Depending on the approach taken in an algorithm, even when run on the same device, the amount of resources used (such as time and memory) can vary significantly. Programmers are particularly interested in how efficiently an algorithm performs on large inputs, since for small inputs even inefficient algorithms may still complete quickly.

Order of Growth

The **order of growth** provides a rough measure of the resources (e.g., time and memory) required by a computational process. This is known as computational complexity, which includes:

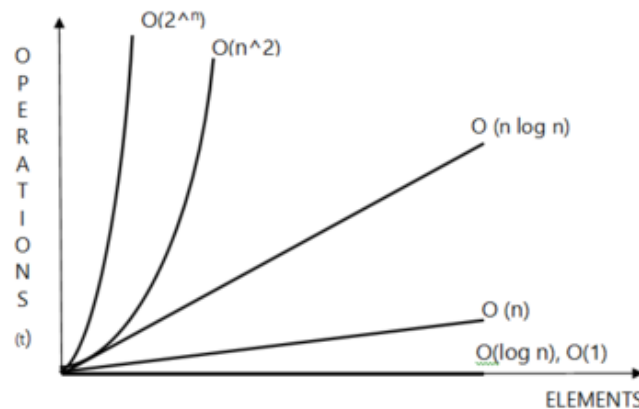
- **Time complexity:** the amount of time taken by an algorithm as a function of input size.
- **Space complexity:** the amount of memory space needed by an algorithm as a function of input size.

Big-O Notation

Big-O notation is used to express the efficiency of an algorithm as input size increases. It is written as **$O(f(n))$** , where:

- n is the size of the input data set.
- $f(n)$ describes the rate of growth in operations or space required.

Only the highest-order term in $f(n)$ is considered, and analysis usually focuses on the **worst-case scenario**.



This graph illustrates the growth of different time complexities using Big-O notation, which is a mathematical way to describe how an algorithm's running time increases as the size of its input (n) grows.

The x-axis represents the number of elements in the input, while the y-axis shows the number of operations required. The curves on the graph represent various common time complexities: $O(1)$ and $O(\log n)$ grow slowly and are considered very efficient; $O(n)$ and $O(n \log n)$ grow moderately and are typical of efficient algorithms like merge sort; $O(n^2)$ represents slower algorithms such as bubble sort, and $O(2^n)$ shows exponential growth, which becomes impractical for large inputs.

This helps compare the efficiency of algorithms and understand which ones scale better with increasing data size.

6.1 Comparison of Sorting Algorithms

The sorting methods discussed can be summarised in the following table:

Sorting algorithm	Time Complexity			Memory usage	Stable
	Best case	Average case	Worst case		
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	In-place	Yes
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	In-place	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	In-place	No
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Not In-place	Yes

Bubble sort is the simplest, but slowest. It works by repeatedly swapping adjacent elements and is only practical for small lists. Best case (already sorted): $O(n)$.

Insertion sort is more efficient than bubble sort. It inserts each element into its correct place. It shifts elements only when it finds an element that is larger than the current element being inserted. This results in fewer swaps and comparisons compared to bubble sort. It performs well on nearly sorted lists. Best case $O(n)$, average/worst case $O(n^2)$.

Quicksort is typically faster than both Bubble and Insertion Sort, especially for large datasets. Its average case is $O(n \log n)$, but worst case is $O(n^2)$ when poor pivot choices lead to unbalanced splits.

Merge Sort has consistently good time complexity of $O(n \log n)$ in all cases and is stable. However, it requires additional memory to store temporary arrays during the merge process, which can be a drawback.

There is no one-size-fits-all sorting method. The best algorithm depends on:

- The number of elements.
- How ordered the data already is.
- Whether memory usage is a concern.
- Whether stability is required (i.e., equal elements retain original order).

Choosing the right sorting algorithm involves balancing **efficiency**, **memory constraints**, **data characteristics**, and **ease of implementation**.

7. Video References

Insertion sort: <https://www.youtube.com/watch?v=O0VbBkUvriI&t=178s>

Bubble sort: <https://www.youtube.com/watch?v=RT-hUXUWQ2I&t=3s>

Quicksort: <https://www.youtube.com/watch?v=7h1s2SojIRw> (4:00 to 9:18)

Merge sort: <https://www.youtube.com/watch?v=Ns7tGNbtvV4>

Computational complexity: <https://www.youtube.com/watch?v=YoZPTyGL2IQ>

8. Appendix

8.1 Analysis of Quicksort Efficiency

Best case

The best case for quicksort occurs when the partitions are as evenly balanced as possible (i.e.: the height of the tree is the shortest). To partition an array from one level to another, $n - 1$ comparisons have to be made as each element is compared to the pivot. Hence, the time complexity of quicksort (best case) is $O(n \log_2 n)$.

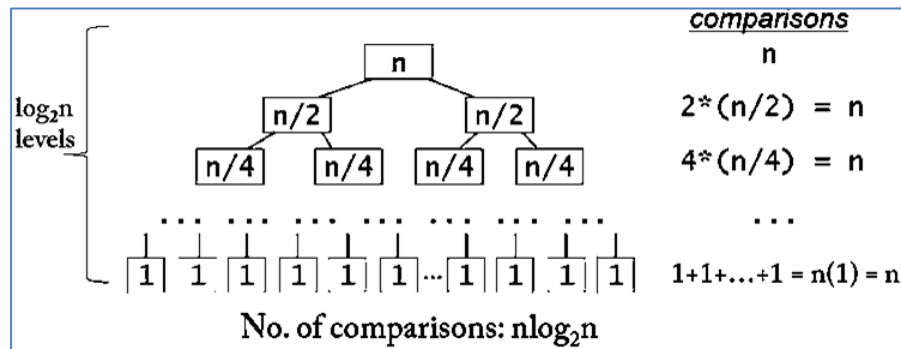


Figure 1: A balanced tree

Worst case

The worst case for quicksort occurs when the partitions are most unbalanced (i.e.: the height of the tree is the tallest). To partition an array from one level to another, $n - 1$ comparisons have to be made as each element is compared to the pivot. Hence, the time complexity of quicksort (worst case) is $O(n^2)$.

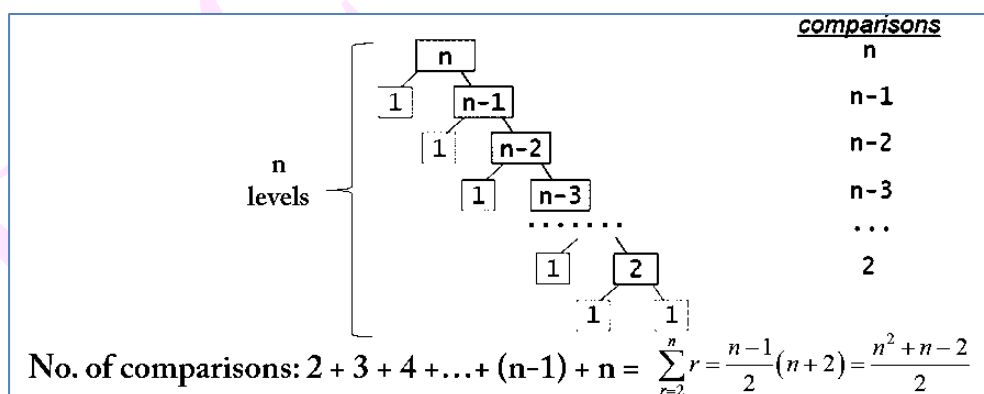
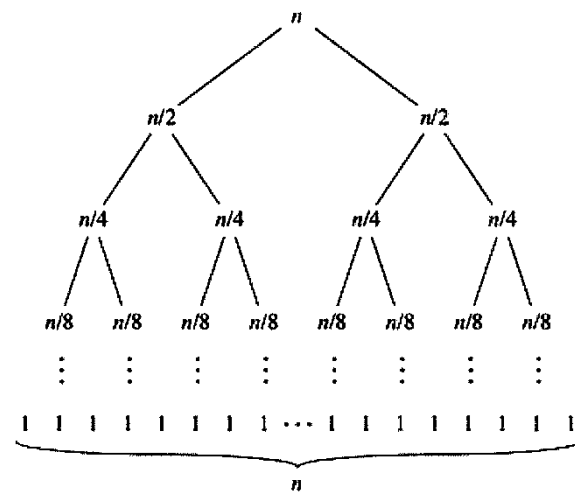


Figure 2: An unbalanced/ skewed tree

8.2 Analysis of Merge Sort Efficiency

Assume the subarrays are of equal size. And let n be the total number of elements in the unsorted array.

Total no of items, n	Total no of levels, m
$1 = 2^{1-1}$	1
$2 = 2^{2-1}$	2
$4 = 2^{3-1}$	3
$8 = 2^{4-1}$	4
$16 = 2^{5-1}$	5
$32 = 2^{6-1}$	6
...	...
$n = 2^{m-1}$	m



$$n = 2^{m-1}$$

$$\log_2 n = (m - 1) \log_2 2$$

$$m = \log_2 n + 1$$

The total number of levels is $\log_2 n + 1$.

At the base case, constant time $O(1)$ is needed to sort each subarray of size 1.

For n elements, the time needed is nc , where c is constant time.

Thus, merging takes linear time $O(n)$.

Time complexity of merge sort

$$= m(nc)$$

$$= (\log_2 n + 1)(nc)$$

$$\approx O(n \log_2 n)$$